The task is to implement a smart servo in the Pikes robot that can open and close a door. A transmission box is located between the servo and the door mechanism. Since the open and close endpoints may vary from robot to robot, intelligent and partially automated firmware must be developed to support both the setup and operation processes.

The system includes a smart servo simulation using an ESP32 board to control the servo. The main development eAort focuses on firmware running on the ESP32. In addition, the virtual servo should be visualized. A gateway, for example based on TCP/IP, MQTT, or a similar protocol, is used to bridge the ESP32 simulation with ROS. ROS is intended to serve as the primary framework in which the ESP32 firmware is integrated and orchestrated. On the ROS side, the servo should be controllable through keyboard inputs.

Task #1 – System Setup

Firmware

- Develop firmware for the ESP32
- Support a 360-degree (continuous-rotation) servo
- Implement smart servo control using an ESP32 board
- Handle the 40:15 transmission ratio on the ESP32 side

Servo Simulation

- Simulate the servo using a framework such as Wokwi (framework is free to choose)

Servo Visualization

- Visualize servo motion within the simulation
- For example, attach a virtual arm to the servo shaft

ROS Integration

- Stream data from the servo simulation to ROS
    - For example, using the Wokwi Gateway

ROS 2 Control

- Use the Resource Manager and implement a controller to:
    - Read values from the servo
    - Write values to the servo
- Control the servo via keyboard input, for example:
    - Left/Right arrow keys to rotate the servo
    - +/- keys to adjust the rotation speed

- Result
    - The servo can be rotated continuously using the arrow keys
    - Show rotation with and without gear transmission

Task #2 – Endpoint Detection

- Extend the ESP32 firmware to simulate randomized endpoints (door open and door
- closed positions)
- The endpoints shall be within a range of 60°
- Implement a stop condition to terminate the endpoint detection process
    - For example, stop when a specified torque threshold is reached
- If torque cannot be simulated directly, implement a function that mimics torque behavior (no physical simulation is required)

Result

- Display the detected endpoint limits in the simulation next to the servo
- Servo movement controlled by the arrow keys shall be restricted by the detected endpoints

Task #3 – Documentation

- Create a system diagram that briefly explains each system component
- Provide instructions for deploying and starting the system
- Record a short video demonstrating Task #1 and Task #2

Optional:

- Feel free to show extensions, graphs, and creative ideas

Submission

- Please send a short video, documentation, and the source files (or a link to GitHub) by July 1.

In case of questions please contact (always both):

michael.wojtynek@pikes.de

eric.kraemer@pikes.de